

SYSTEM DESIGN DOCUMENT (SDD)

CivicPulse

Public Service Workflow Platform

Version: 1.0

Status: Draft

Architecture Style: Modular Monolith

Prepared By: Rizqi

1. System Overview

CivicPulse adalah platform GovTech berbasis web yang memungkinkan masyarakat melaporkan permasalahan fasilitas publik dan memungkinkan instansi pemerintah memproses laporan melalui workflow terstruktur, transparan, dan terukur.

Platform dirancang menggunakan pendekatan Modular Monolith untuk menjaga keseimbangan antara kecepatan pengembangan, maintainability, dan scalability.

2. Architectural Goals

Tujuan arsitektur sistem:

- Maintainable
- Scalable
- Secure
- Observable
- Testable
- Production Ready

Prinsip utama:

- Separation of Concerns
 - Single Responsibility
 - Feature Driven Architecture
 - Domain Oriented Design
 - API First Development
-

3. High-Level Architecture

User Browser



Next.js Frontend



Route Handlers API



Application Services



Repositories



PostgreSQL

Supporting Services

Redis

BullMQ

Object Storage

Sentry

OpenTelemetry

4. Architecture Style

Pattern:

Modular Monolith

Alasan:

- Lebih sederhana dibanding Microservices

- Mudah di-deploy
 - Mudah dikembangkan oleh tim kecil
 - Cocok untuk MVP dan kompetisi
 - Dapat berkembang menjadi Distributed System di masa depan
-

5. System Context Diagram

Citizen



Web Application



CivicPulse Platform



PostgreSQL



Object Storage



Notification Service



Government Officer

6. Core Modules

Sistem dibagi berdasarkan domain bisnis.

Modules:

- Authentication
- Users
- Agencies
- Reports
- Attachments
- Notifications

- Analytics
- Audit Logs
- Dashboard

Setiap module memiliki:

- Components
 - Services
 - Repositories
 - Validators
 - Types
-

7. Module Architecture

Example:

Reports Module

reports/

components/

services/

repositories/

validators/

types/

api/

Responsibilities

Components

UI rendering

Services

Business logic

Repositories

Database access

Validators

Request validation

API

Route handlers

8. Frontend Architecture

Framework:

Next.js 16

Pattern:

App Router

Server Components

Client Components

Server Actions (optional)

UI Layer

Pages

↓

Feature Components

↓

Shared Components

↓

UI Components

Technology

- TypeScript
- TailwindCSS

- shadcn/ui
 - TanStack Query
 - Zustand
-

9. Backend Architecture

Pattern:

Layered Architecture

Presentation Layer

↓

Application Layer

↓

Domain Layer

↓

Infrastructure Layer

Presentation Layer

Responsibilities:

- Request handling
- Response formatting
- Validation

Files:

route.ts

Application Layer

Responsibilities:

- Business workflows
- Orchestration

Examples:

CreateReportService

AssignReportService

ResolveReportService

Domain Layer

Responsibilities:

- Domain rules
- Domain entities

Examples:

Report

User

Agency

Infrastructure Layer

Responsibilities:

- Database
 - Storage
 - Queue
 - External services
-

10. Authentication Architecture

Solution:

Better Auth

Authentication Flow

User Login

↓

Credential Validation

↓

Session Creation



Cookie Storage



Authenticated Access

Protected Routes

Middleware



Role Validation



Allow / Deny

11. Authorization Architecture

RBAC Model

Roles:

CITIZEN

OFFICER

SUPER_ADMIN

Permission Matrix

Citizen

- Create Report
- View Own Reports

Officer

- View Assigned Reports
- Update Status

- Add Resolution

Super Admin

- Full Access
-

12. Report Workflow Architecture

Report Lifecycle

DRAFT



SUBMITTED



ASSIGNED



IN_PROGRESS



RESOLVED



CLOSED

Workflow Rules

Only Officer may move:

ASSIGNED



IN_PROGRESS



RESOLVED

Only Citizen may verify closure.

13. Attachment Architecture

Purpose

Store evidence photos.

Storage Strategy

Metadata

↓

PostgreSQL

Files

↓

Object Storage

Supported Formats

JPG

PNG

WEBP

Maximum File Size

10 MB

14. Notification Architecture

Events

Report Created

Report Assigned

Report Updated

Report Resolved

Flow

Event Triggered

↓

Queue Job

↓

Notification Service

↓

Email / In-App Notification

Technology

Redis

BullMQ

15. Audit Log Architecture

Purpose

Track all critical actions.

Events

User Login

Report Created

Status Changed

Role Changed

Agency Updated

Stored Data

Actor

Action

Resource

Timestamp

Old Value

New Value

16. Analytics Architecture

Data Sources

Reports

Users

Agencies

Audit Logs

Metrics

Total Reports

Open Reports

Resolved Reports

Resolution Rate

Average Response Time

Average Resolution Time

17. Caching Strategy

Technology

Redis

Cache Targets

Dashboard Statistics

Analytics

Frequently Accessed Reports

Cache Pattern

Cache Aside

18. Database Architecture

Primary Database

PostgreSQL

ORM

Prisma ORM

Core Tables

Users

Agencies

Reports

Attachments

Notifications

AuditLogs

ReportHistory

Sessions

19. Search Architecture

Features

Keyword Search

Status Filter

Category Filter

Location Filter

Date Filter

Version 1

PostgreSQL Full Text Search

Future

Elasticsearch

20. Geolocation Architecture

Stored Data

Latitude

Longitude

Address

District

City

Province

Features

Map Visualization

Regional Filtering

Heatmaps

Location Analytics

21. Security Architecture

Authentication

Better Auth

Authorization

RBAC

Validation

Zod

Security Controls

Password Hashing

Rate Limiting

Input Sanitization

CSRF Protection

Session Validation

Audit Logging

Secure Headers

22. Observability Architecture

Logging

Pino

Monitoring

OpenTelemetry

Error Tracking

Sentry

Tracked Metrics

Request Count

Response Time

Error Rate

Database Latency

Queue Performance

23. Deployment Architecture

Production Environment

Internet

↓

Reverse Proxy

↓

Next.js Application

↓

PostgreSQL

↓

Redis

↓

Object Storage

Deployment Stack

Docker

Docker Compose

Coolify

24. Scalability Strategy

Phase 1

Single VPS

Single Database

Phase 2

Dedicated Database

Dedicated Redis

CDN

Phase 3

Horizontal Scaling

Load Balancer

Read Replicas

Distributed Cache

25. Disaster Recovery

Backup Frequency

Daily

Retention

30 Days

Recovery Targets

RPO

24 Hours

RTO

4 Hours

26. Testing Strategy

Unit Testing

Vitest

Integration Testing

Testing Library

E2E Testing

Playwright

Coverage Target

Minimum 80%

27. Technical Decisions

Architecture

Modular Monolith

Database

PostgreSQL

ORM

Prisma

Frontend

Next.js

Authentication

Better Auth

Caching

Redis

Queue

BullMQ

Storage

S3 Compatible Storage

Monitoring

Sentry

Deployment

Docker

28. Future Evolution

Current

Modular Monolith

↓

Event Driven Modules

↓

Service Extraction

↓

Microservices (If Needed)

No premature microservices.